Machine Learning

Regresión Líneal Multiple

Autor: Anderson Jawarlan Vargas Ruiz

Profesor: Danny Aldemar Jimenez Martin

Universidad: Fundación universitaria UNICAFAM

Facultad: Ingeniería

Curso: Inteligencia artificial II 5to Semestre

Bogotá D.C

20 Febrero 2026

Resumen

Este documento contiene el desarrollo de ejercicios prácticos de machine learning y regresión lineal multiple o multi-variable, incluyendo representaciones gráficas mediante graficos de dispersión y mapa de calor con operaciones sobre modelo de entrenamiento y error cuadratico medio. Se valida la disponibilidad de los dataset mencionados en clase, se crea una hipotesis aleatoria para la verificación del entrenamiento y se grafica.

Herramientas utilizadas

Librerias

In [1]:
```
import pandas as pd

import matplotlib.pyplot as plt

import numpy as np

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split #

import seaborn as sns

#Pendiente estudiar
#plt.style.use("seaborn-v0_8-whitegrid")
```

Mostrar por completo los datos de los DataFrame

In [2]:
```
pd.set_option("display.max_rows", None) # Mostrar todas las filas
pd.set_option("display.max_columns", None) # Mostrar todas las columnas
pd.set_option("display.max_colwidth", None)# Ajustar el ancho de las columnas (para no cort
pd.set_option("display.width", None) # Evitar que se corte horizontalmente
```

Validación de Datasets disponibles de SEABORN

In [3]:
```
Sets_Disponibles = pd.DataFrame(columns=["DataSet","Campos"])
for i in sns.get_dataset_names():
    for j in sns.load_dataset(i).columns:
        Sets_Disponibles.loc[len(Sets_Disponibles)] = [i,j]

Sets_Disponibles #Validación de Datassets Disponibles
```

Out[3]:

	DataSet	Campos
0	anagrams	subidr
1	anagrams	attnr
2	anagrams	num1
3	anagrams	num2
4	anagrams	num3
5	anscombe	dataset
6	anscombe	x
7	anscombe	y
8	attention	Unnamed: 0
9	attention	subject
10	attention	attention
11	attention	solutions
12	attention	score
13	brain_networks	network
14	brain_networks	1
15	brain_networks	1.1
16	brain_networks	2
17	brain_networks	2.1
18	brain_networks	3
19	brain_networks	3.1
20	brain_networks	4
21	brain_networks	4.1
22	brain_networks	5
23	brain_networks	5.1
24	brain_networks	6
25	brain_networks	6.1
26	brain_networks	6.2
27	brain_networks	6.3
28	brain_networks	7
29	brain_networks	7.1
30	brain_networks	7.2
31	brain_networks	7.3
32	brain_networks	7.4
33	brain_networks	7.5
34	brain_networks	8
35	brain_networks	8.1
36	brain_networks	8.2

	DataSet	Campos
37	brain_networks	8.3
38	brain_networks	8.4
39	brain_networks	8.5
40	brain_networks	9
41	brain_networks	9.1
42	brain_networks	10
43	brain_networks	10.1
44	brain_networks	11
45	brain_networks	11.1
46	brain_networks	12
47	brain_networks	12.1
48	brain_networks	12.2
49	brain_networks	12.3
50	brain_networks	12.4
51	brain_networks	13
52	brain_networks	13.1
53	brain_networks	13.2
54	brain_networks	13.3
55	brain_networks	13.4
56	brain_networks	13.5
57	brain_networks	14
58	brain_networks	14.1
59	brain_networks	15
60	brain_networks	15.1
61	brain_networks	16
62	brain_networks	16.1
63	brain_networks	16.2
64	brain_networks	16.3
65	brain_networks	16.4
66	brain_networks	16.5
67	brain_networks	16.6
68	brain_networks	16.7
69	brain_networks	17
70	brain_networks	17.1
71	brain_networks	17.2
72	brain_networks	17.3
73	brain_networks	17.4

	DataSet	Campos
74	brain_networks	17.5
75	brain_networks	17.6
76	car_crashes	total
77	car_crashes	speeding
78	car_crashes	alcohol
79	car_crashes	not_distracted
80	car_crashes	no_previous
81	car_crashes	ins_premium
82	car_crashes	ins_losses
83	car_crashes	abbrev
84	diamonds	carat
85	diamonds	cut
86	diamonds	color
87	diamonds	clarity
88	diamonds	depth
89	diamonds	table
90	diamonds	price
91	diamonds	x
92	diamonds	y
93	diamonds	z
94	dots	align
95	dots	choice
96	dots	time
97	dots	coherence
98	dots	firing_rate
99	dowjones	Date
100	dowjones	Price
101	exercise	Unnamed: 0
102	exercise	id
103	exercise	diet
104	exercise	pulse
105	exercise	time
106	exercise	kind
107	flights	year
108	flights	month
109	flights	passengers
110	fmri	subject

	DataSet	Campos
111	fmri	timepoint
112	fmri	event
113	fmri	region
114	fmri	signal
115	geyser	duration
116	geyser	waiting
117	geyser	kind
118	glue	Model
119	glue	Year
120	glue	Encoder
121	glue	Task
122	glue	Score
123	healthexp	Year
124	healthexp	Country
125	healthexp	Spending_USD
126	healthexp	Life_Expectancy
127	iris	sepal_length
128	iris	sepal_width
129	iris	petal_length
130	iris	petal_width
131	iris	species
132	mpg	mpg
133	mpg	cylinders
134	mpg	displacement
135	mpg	horsepower
136	mpg	weight
137	mpg	acceleration
138	mpg	model_year
139	mpg	origin
140	mpg	name
141	penguins	species
142	penguins	island
143	penguins	bill_length_mm
144	penguins	bill_depth_mm
145	penguins	flipper_length_mm
146	penguins	body_mass_g
147	penguins	sex

	DataSet	Campos
148	planets	method
149	planets	number
150	planets	orbital_period
151	planets	mass
152	planets	distance
153	planets	year
154	seaice	Date
155	seaice	Extent
156	taxis	pickup
157	taxis	dropoff
158	taxis	passengers
159	taxis	distance
160	taxis	fare
161	taxis	tip
162	taxis	tolls
163	taxis	total
164	taxis	color
165	taxis	payment
166	taxis	pickup_zone
167	taxis	dropoff_zone
168	taxis	pickup_borough
169	taxis	dropoff_borough
170	tips	total_bill
171	tips	tip
172	tips	sex
173	tips	smoker
174	tips	day
175	tips	time
176	tips	size
177	titanic	survived
178	titanic	pclass
179	titanic	sex
180	titanic	age
181	titanic	sibsp
182	titanic	parch
183	titanic	fare
184	titanic	embarked

	DataSet	Campos
185	titanic	class
186	titanic	who
187	titanic	adult_male
188	titanic	deck
189	titanic	embark_town
190	titanic	alive
191	titanic	alone

Solución del Taller

In [4]:
```
pd.DataFrame(Sets_Disponibles["Campos"][Sets_Disponibles["DataSet"]=="car_crashes"])
```

Out[4]:

	Campos
76	total
77	speeding
78	alcohol
79	not_distracted
80	no_previous
81	ins_premium
82	ins_losses
83	abbrev

He seleccionado el dataset denominado car_crashes, es uno de los tantos que me llamo la atención, en este ejercicio entrenaremos nuestro modelo con los datos de accidentes de carros, validaremos los campos y seleccionaremos para nuestro modelo de regresion lineal multivarieble.

In [5]:
```
data = sns.load_dataset("car_crashes")
data.head(5) #Primeros Registros del data set para corroborar su comportamiento
```

Out[5]:

	total	speeding	alcohol	not_distracted	no_previous	ins_premium	ins_losses	abbrev
0	18.8	7.332	5.640	18.048	15.040	784.55	145.08	AL
1	18.1	7.421	4.525	16.290	17.014	1053.48	133.93	AK
2	18.6	6.510	5.208	15.624	17.856	899.47	110.35	AZ
3	22.4	4.032	5.824	21.056	21.280	827.34	142.39	AR
4	12.0	4.200	3.360	10.920	10.680	878.41	165.63	CA

In [6]:
```
campos_es = {
    'total':'Total',
    'speeding':'Exceso De Velocidad',
    'alcohol':'Alcohol',
    'not_distracted':'No Distraído',
    'no_previous':'Sin Antecedentes',
    'ins_premium':'INS Premium',
    'ins_losses':'INS_Pérdidas',
    'abbrev':'Abbrev'
```

```
}
Datos = data.rename(columns=campos_es)
```

In [7]:
```
print(f"Las filas del dataset: {Datos.shape[0]}, las columnas del dataset {Datos.shape[1]}"
Datos.describe()
```

```
Las filas del dataset: 51, las columnas del dataset 8
```

Out[7]:

	Total	Exceso De Velocidad	Alcohol	No Distraído	Sin Antecedentes	INS Premium	INS_Pérdidas
count	51.000000	51.000000	51.000000	51.000000	51.000000	51.000000	51.000000
mean	15.790196	4.998196	4.886784	13.573176	14.004882	886.957647	134.493137
std	4.122002	2.017747	1.729133	4.508977	3.764672	178.296285	24.835922
min	5.900000	1.792000	1.593000	1.760000	5.900000	641.960000	82.750000
25%	12.750000	3.766500	3.894000	10.478000	11.348000	768.430000	114.645000
50%	15.600000	4.608000	4.554000	13.857000	13.775000	858.970000	136.050000
75%	18.500000	6.439000	5.604000	16.140000	16.755000	1007.945000	151.870000
max	23.900000	9.450000	10.038000	23.661000	21.280000	1301.520000	194.780000

Matriz de correlación

In [8]:
```
Matriz_Cor = Datos.corr(numeric_only=True)
Matriz_Cor
```

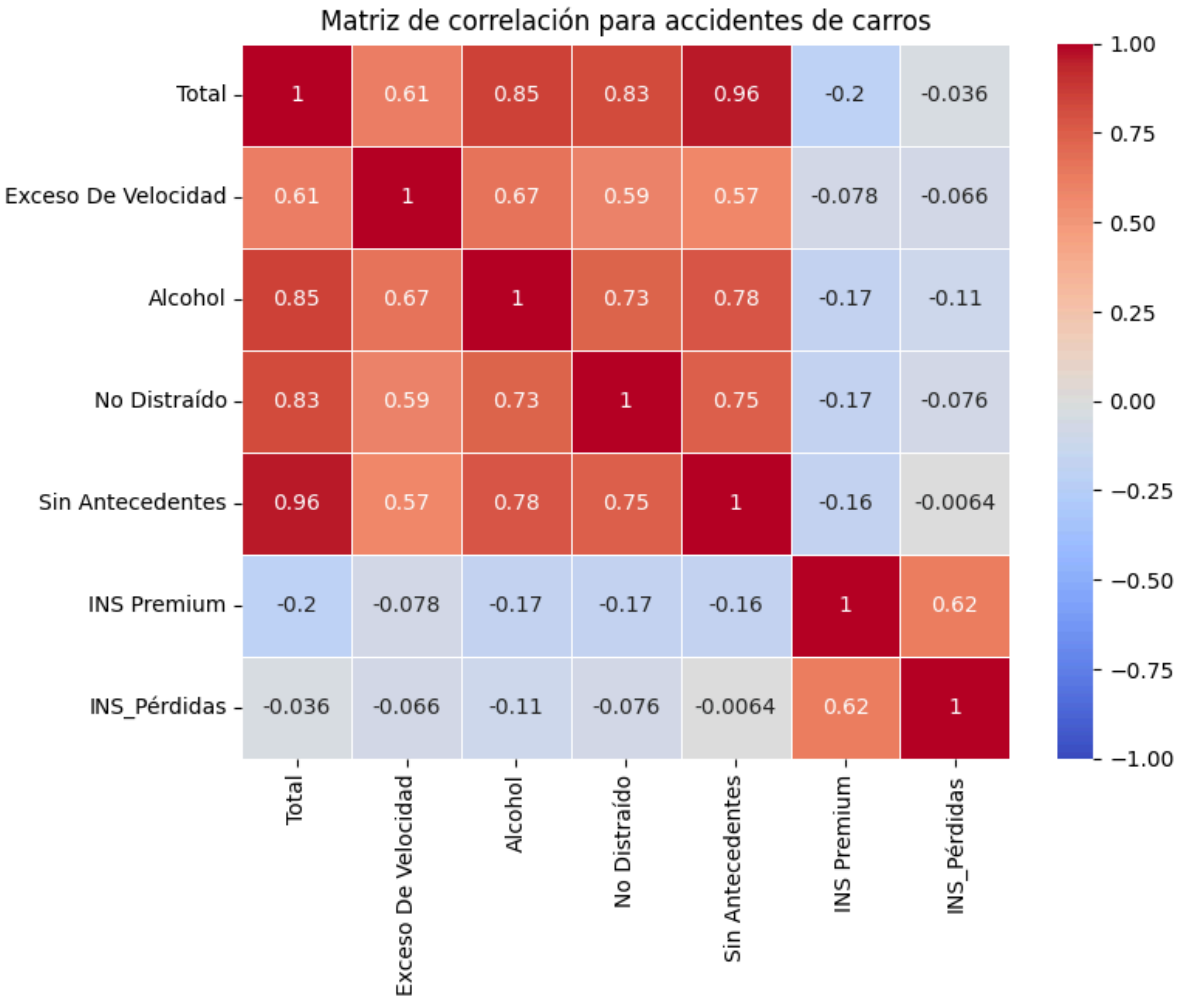
Out[8]:

	Total	Exceso De Velocidad	Alcohol	No Distraído	Sin Antecedentes	INS Premium	INS_Pérdidas
Total	1.000000	0.611548	0.852613	0.827560	0.956179	-0.199702	-0.036011
Exceso De Velocidad	0.611548	1.000000	0.669719	0.588010	0.571976	-0.077675	-0.065928
Alcohol	0.852613	0.669719	1.000000	0.732816	0.783520	-0.170612	-0.112547
No Distraído	0.827560	0.588010	0.732816	1.000000	0.747307	-0.174856	-0.075970
Sin Antecedentes	0.956179	0.571976	0.783520	0.747307	1.000000	-0.156895	-0.006359
INS Premium	-0.199702	-0.077675	-0.170612	-0.174856	-0.156895	1.000000	0.623116
INS_Pérdidas	-0.036011	-0.065928	-0.112547	-0.075970	-0.006359	0.623116	1.000000

Mapa de Calor

In [9]:
```
plt.figure(figsize=(8,6))
sns.heatmap(
    Matriz_Cor,
    annot=True,
    cmap="coolwarm",
    linewidths=0.4,
    center=0,
    vmin=-1,
    vmax=1
)
```

```
plt.title("Matriz de correlación para accidentes de carros")
plt.show()
```



Pedicciones

Como podemos observar en nuestro mapa de calor, tenemos una relación fuerte entre el Total y % accidentes donde los conductores no tienen antecedentes, despues una correlación mas fuerte entre el % total y % de conductores accidentados por el alcohol, junto con conductores que no se distraen en las vias, en este orden de ideas, entraremos a realizar las predicciones del Total de accidentes con respecto a sin antecedentes, Alcohol y no distraidos. Los datos estimados los predeciremos con el 80% dejando así el 20% para la validación del modelo.

In [10]:
```
x = Datos[["Sin Antecedentes","Alcohol","No Distraído"]]
y = Datos["Total"]
x_ent, x_pru, y_ent, y_pru = train_test_split(x,y, test_size=0.2, random_state=42)
```

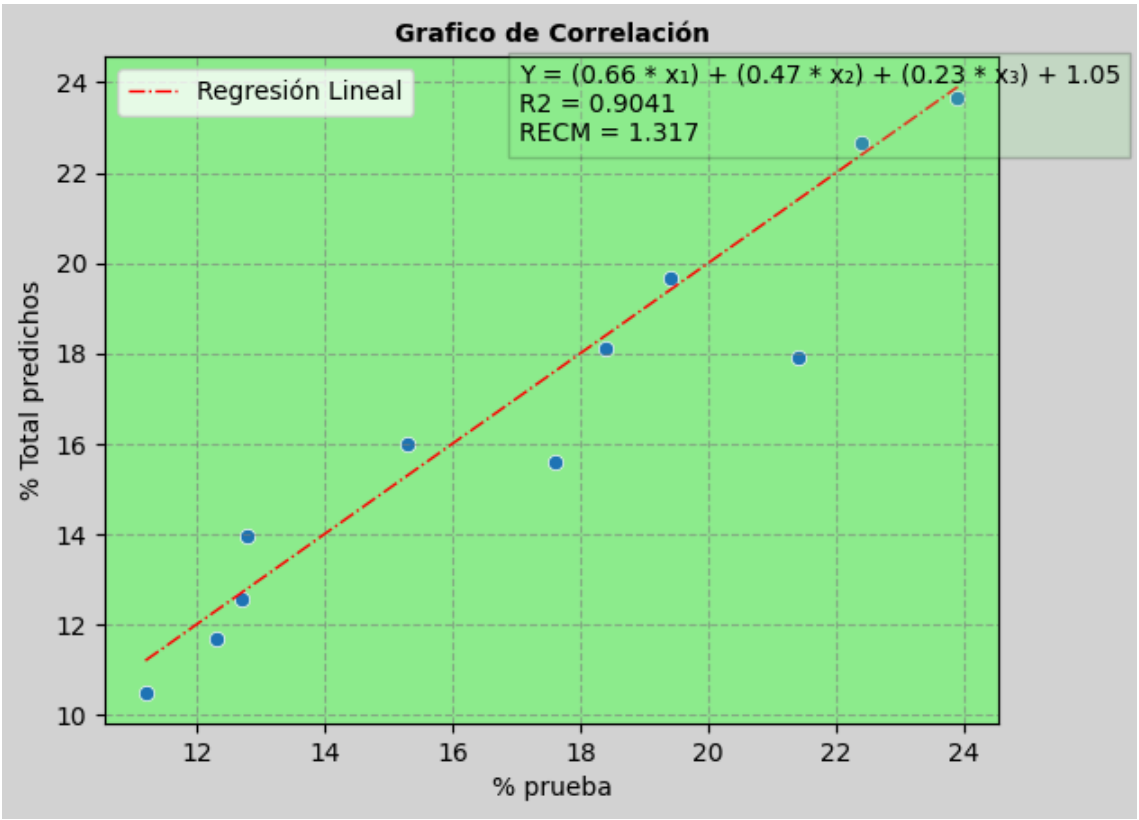
In [11]:
```
modelo = LinearRegression()
modelo.fit(x_ent,y_ent)

b = modelo.intercept_
m = modelo.coef_

y_pred = modelo.predict(x_pru)
ini_r2 = r2_score(y_pru, y_pred)
recm = mean_squared_error(y_pru,y_pred)**(1/2)

print("Coheficientes del modelo:", m)
print("Punto de intersepción:", b)
```

```
print("R2 del modelo:", ini_r2)
print("Raiz del erro cuadratico medio:",recm)
```

```
Coheficientes del modelo: [0.66013553 0.46595574 0.2298896 ]
Punto de intersepción: 1.0485200745240295
R2 del modelo: 0.9040615643328077
Raiz del erro cuadratico medio: 1.3172462906377977
```

In [12]:

```
fig, ax = plt.subplots()

# Tu scatterplot y línea
sns.scatterplot(x=y_pru, y=y_pred, ax=ax)
line = [y_pru.min(), y_pru.max()]
ax.plot(line, line, "-.r", label="Regresión Lineal", linewidth=1)

ax.text(
    y_pru.mean(), y_pru.max()-1.2,
    f"Y = ({m[0]:.2f} * x\u2081) + ({m[1]:.2f} * x\u2082) + ({m[2]:.2f} * x\u2083) + {b:.2f
    fontsize=10, color="k",
    bbox=dict(facecolor="lightgreen", alpha=0.2)
)

ax.set_title("Grafico de Correlación", fontsize=10, fontweight="bold")
ax.set_xlabel("% prueba", fontsize=10)
ax.set_ylabel("% Total predichos", fontsize=10)

# Cuadrícula y leyenda
ax.grid(True, color="gray", linestyle="--", alpha=0.6)
ax.legend(loc="best", fontsize=10)

# Fondos personalizados
ax.set_facecolor("lightgreen")
fig.patch.set_facecolor("lightgray")

plt.show()
```



Hipotesis para predecir en nuestro modelo multi-variable

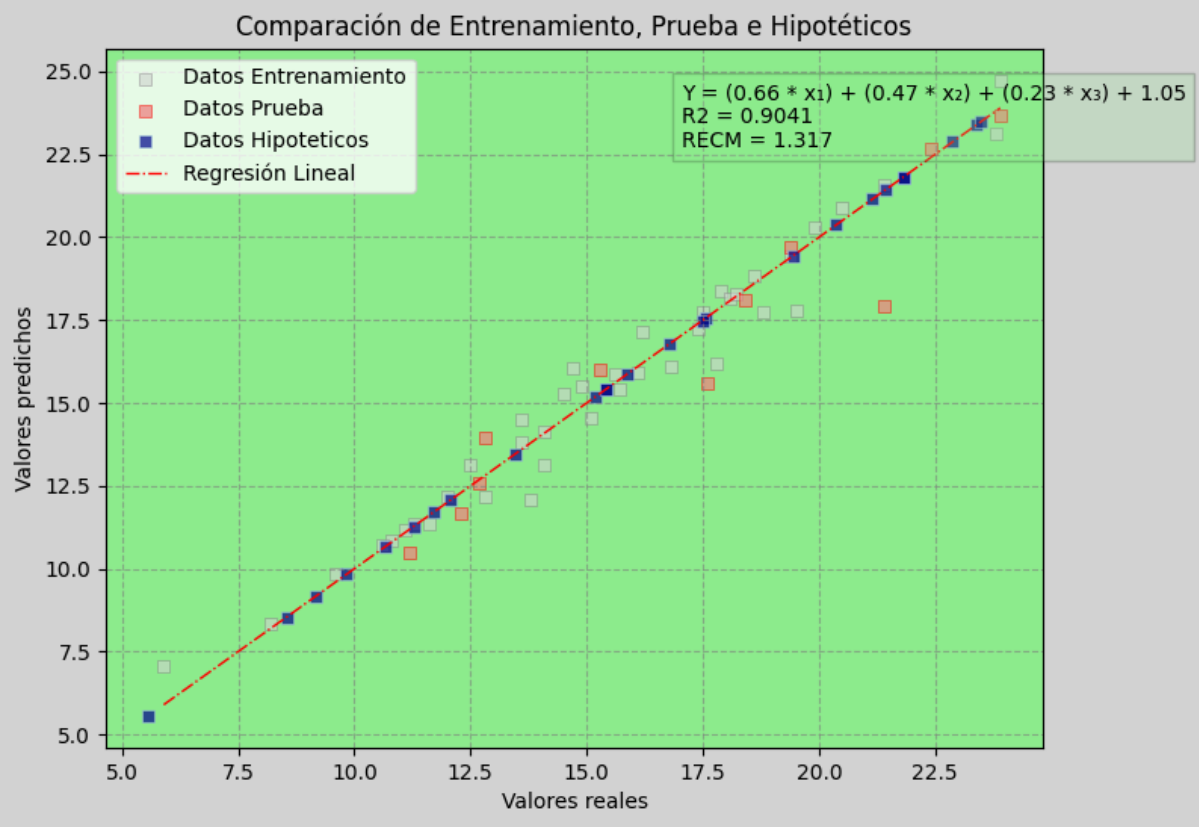
In [13]:

```
#Generamos 100 Datos aleatorios para validación de nuestro modelo
rangos = {
    "Sin Antecedentes": (x_ent["Sin Antecedentes"].min() - 3, x_ent["Sin Antecedentes"].max
    "Alcohol": (x_ent["Alcohol"].min() - 3, x_ent["Alcohol"].max() + 5),
    "No Distraído": (x_ent["No Distraído"].min() - 3, x_ent["No Distraído"].max() + 5),
}

np.random.seed(42)  # para reproducibilidad

# Hipotesis
Datos_Hipo = {
    "Sin Antecedentes": np.random.uniform(rangos["Sin Antecedentes"][0], rangos["Sin Antece
    "Alcohol": np.random.uniform(rangos["Alcohol"][0], rangos["Alcohol"][1], 25),
    "No Distraído": np.random.uniform(rangos["No Distraído"][0], rangos["No Distraído"][1],
}

# Pandas con datos encontrados de acuerdo al modelo.
Datos_Hipo_Pd = pd.DataFrame(Datos_Hipo)

# Predicción Mediante f(x) = (m1*x1) + (m2*x2) + (m3*x3) + b
Datos_Hipo_Pd["f(x)"] = (
    (m[0] * Datos_Hipo_Pd["Sin Antecedentes"]) + (m[1] * Datos_Hipo_Pd["Alcohol"]) + (m[2]
)

#PRedicción del Modelo
Datos_Hipo_Pd["Predicción Modelo"] = modelo.predict(pd.DataFrame(Datos_Hipo))

#Datos para graficar
y_H = np.array(Datos_Hipo_Pd["f(x)"])

#Tabla de Muestra
Datos_Hipo_Pd
```

Out[13]:

	Sin Antecedentes	Alcohol	No Distraído	f(x)	Predicción Modelo
0	11.441762	11.505219	27.861714	20.367675	20.367675
1	24.581990	1.876635	22.751715	23.380777	23.380777
2	19.593854	7.049585	27.071093	23.491277	23.491277
3	16.553005	8.335258	25.897168	21.813098	21.813098
4	6.458161	-0.643123	18.094214	9.171786	9.171786
5	6.457611	8.584075	26.607933	15.428105	15.428105
6	4.224655	1.397269	4.707494	5.570634	5.570634
7	22.654013	-0.337227	7.532234	17.577689	17.577689
8	16.609029	14.197423	3.570528	19.448928	19.448928
9	19.048303	14.472819	10.931356	22.879680	22.879680
10	3.369450	11.887094	12.596050	11.707375	11.707375
11	25.019764	3.602373	9.512781	21.430391	21.430391
12	21.884687	0.199218	24.160393	21.142429	21.142429
13	7.742606	9.845212	11.757121	13.449962	13.449962
14	7.046700	5.831308	9.764678	10.662227	10.662227
15	7.082723	0.599919	16.643510	9.829783	9.829783
16	9.838549	6.736184	6.085348	12.081017	12.081017
17	14.867595	-0.841481	23.462934	15.864940	15.864940
18	12.750938	13.546774	4.341116	16.776042	16.776042
19	9.541772	2.848637	28.316402	15.184368	15.184368
20	16.853917	9.488179	22.675820	21.808396	21.808396
21	6.081297	3.719089	7.604049	8.544023	8.544023
22	9.562651	7.145519	2.527116	11.271619	11.271619
23	11.255248	7.583651	23.811511	17.486174	17.486174
24	13.301132	1.632932	20.957504	15.407856	15.407856

In [14]:

```
fig, ax = plt.subplots(figsize=(8,6))

# Datos de entrenamiento
sns.scatterplot(x=y_ent, y=modelo.predict(x_ent), ax=ax,
                label="Datos Entrenamiento", color="lightgray", alpha=0.6,
                marker="s", edgecolor="gray"
                )

# Datos de prueba
sns.scatterplot(x=y_pru, y=y_pred, ax=ax,
                label="Datos Prueba", color="lightcoral", alpha=0.7,
                marker="s", edgecolor="red"
                )

# Datos hipotéticos
sns.scatterplot(x=Datos_Hipo_Pd["f(x)"], y=modelo.predict(pd.DataFrame(Datos_Hipo)), ax=ax,
                label="Datos Hipoteticos", color="darkblue", alpha=0.7,
                marker="s", edgecolor="lightblue"
                )
```

```python
# Línea de referencia
line = [y.min(), y.max()]
ax.plot(line, line, "-.r", label="Regresión Lineal", linewidth=1)

ax.text(
    y_pru.mean(), y_pru.max()-1.2,
    f"Y = ({m[0]:.2f} * x\u2081) + ({m[1]:.2f} * x\u2082) + ({m[2]:.2f} * x\u2083) + {b:.2f
    fontsize=10, color="k",
    bbox=dict(facecolor="lightgreen", alpha=0.2)
)

ax.set_title("Comparación de Entrenamiento, Prueba e Hipotéticos")
ax.set_xlabel("Valores reales")
ax.set_ylabel("Valores predichos")
ax.legend()
#ax.grid(True, linestyle="-", alpha=0.6)

# Cuadrícula y leyenda
ax.grid(True, color="gray", linestyle="--", alpha=0.6)
ax.legend(loc="best", fontsize=10)

# Fondos personalizados
ax.set_facecolor("lightgreen")
fig.patch.set_facecolor("lightgray")

plt.show()
```